

Secventa de suma maxima (Maximum Subarray)

1. Cerinta (formulare tipica)

Se da un vector $a[0..n-1]$ (poate contine si valori negative). Sa se determine secventa (subvectorul) cu suma maxima. De obicei se cere: valoarea sumei maxime, sau si pozitiile (inceput/sfarsit).

2. Ce inseamna secventa (re-amintire)

O secventa este $a[i..j]$, cu elemente consecutive. Suma secventei este $a[i] + a[i+1] + \dots + a[j]$.

3. Exemplu intuitiv (de ce nu e trivial)

$a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ Secventa de suma maxima este $[4, -1, 2, 1]$ cu suma 6.

4. Metoda corecta la bac: algoritmul lui Kadane ($O(n)$)

Idee: parcurgem vectorul o singura data si mentinem: - suma_curenta: cea mai buna suma a unei secvente care se termina exact pe pozitia curenta - suma_maxima: cea mai buna suma gasita pana acum Cand adaugarea lui $a[i]$ la suma_curenta nu ajuta (suma_curenta devine mai mica decat $a[i]$ singur), incepem o secventa noua de la i .

5. Exemplu pas cu pas (Kadane)

$a = [-2, 1, -3, 4, -1, 2, 1]$ Pornim cu: suma_curenta = $a[0] = -2$, suma_maxima = -2 $i=1$, $a[i]=1$: $\max(1, -2+1=-1) \Rightarrow$ suma_curenta=1; suma_maxima= $\max(-2,1)=1$ $i=2$, $a[i]=-3$: $\max(-3, 1+(-3)=-2) \Rightarrow$ suma_curenta=-2; suma_maxima=1 $i=3$, $a[i]=4$: $\max(4, -2+4=2) \Rightarrow$ suma_curenta=4; suma_maxima=4 $i=4$, $a[i]=-1$: $\max(-1, 4+(-1)=3) \Rightarrow$ suma_curenta=3; suma_maxima=4 $i=5$, $a[i]=2$: $\max(2, 3+2=5) \Rightarrow$ suma_curenta=5; suma_maxima=5 $i=6$, $a[i]=1$: $\max(1, 5+1=6) \Rightarrow$ suma_curenta=6; suma_maxima=6

6. Implementare minima (suma maxima)

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    long long x;
    cin >> x;
    long long cur = x, best = x;
    for(int i=1;i<n;i++){
        cin >> x;
        if(cur + x < x) cur = x;
        else cur = cur + x;
        if(cur > best) best = cur;
    }
    cout << best << "\n";
    return 0;
}
```

7. Implementare (suma maxima + pozitiile i..j)

La bac se poate cere si secventa efectiv, adica indicii de inceput si sfarsit. Se tine minte inceputul curent si cel mai bun interval gasit.

```
#include <iostream>
using namespace std;

int main(){
    int n; cin >> n;
    long long x;
    cin >> x;
    long long cur = x, best = x;
    int startCur = 0, startBest = 0, endBest = 0;

    for(int i=1;i<n;i++){
        cin >> x;
        if(cur + x < x){
            cur = x;
            startCur = i;
        }else{
            cur = cur + x;
        }
        if(cur > best){
            best = cur;
            startBest = startCur;
            endBest = i;
        }
    }
    cout << best << " " << startBest << " " << endBest << "\n";
    return 0;
}
```

8. Cazuri speciale importante

- Daca toate elementele sunt negative, secventa de suma maxima este un singur element: cel mai mare (cel mai putin negativ). - Daca exista mai multe secvente cu aceeasi suma maxima, enuntul poate cere prima/ultima; daca nu spune, orice este acceptata. - Pentru $n=1$, raspunsul este elementul unic.

Exemplu (toate negative):

$a = [-5, -2, -8] \rightarrow$ suma maxima = -2, secventa = [-2]

9. Legatura cu sume partiale (comparatie)

Se poate rezolva si cu sume partiale in $O(n^2)$ (testand toate intervalele), dar Kadane este $O(n)$ si este varianta recomandata.

10. Complexitate

Kadane: $O(n)$ timp, $O(1)$ memorie suplimentara.

11. Greseli frecvente

- initializarea gresita cu 0 (duce la raspuns gresit cand toate sunt negative) - confundarea secventei cu multime de elemente (trebuie consecutive) - actualizarea gresita a indicilor start/end

12. Checklist de BAC

- stii definitia secventei $a[i..j]$ - stii ideea 'cea mai buna secventa care se termina la i ' -
tratezi corect cazul tuturor numerelor negative - poti obtine si indicii intervalului, daca se
cere